

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 3

The Command Line: Shell, Pipes and Text Filters

Cosneanu Corina, FAF-241

Checked by:
Reitman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 ALGORITHM ANALYSIS AND OBJECTIVES **1**
1.1 Objective 1
1.2 Tasks 1
1.3 Theoretical Notes 1

2 IMPLEMENTATION AND PRACTICAL EXECUTION **2**
2.1 Part 1: Move fast, find help 2
2.2 Part 2: What the shell does to your line 2
2.3 Part 3: Redirection, chaining and jobs 2
2.4 Part 4: Answer questions with one-liners 3
2.4.1 Question 1: Number of Failed Requests 3
2.4.2 Question 2: Different Requested Pages 3
2.4.3 Question 3: Number of Requests for Each Page 3
2.4.4 Question 4: Users with the Most Failed Requests 3
2.4.5 Question 5: Requests Made by user3 3
2.4.6 Question 6: Last Three Failed Requests 3
2.4.7 Question 7: Login Shells from /etc/passwd 3
2.4.8 Question 8: Comparing Line Counts 3
2.5 Part 5: First Automated Bash Script 4

3 CONCLUSIONS **5**

1 ALGORITHM ANALYSIS AND OBJECTIVES

1.1 Objective

The objective of this laboratory work is to master the Linux command-line shell, understand how the shell processes command lines prior to execution, manage standard input/output streams through redirection and pipes, and build efficient text-processing pipelines using standard Unix filters.

1.2 Tasks

1. Study basic shell navigation, use autocompletion (`Tab`), command history, and differentiate between shell built-ins and external binaries.
2. Analyze pre-execution shell mechanisms including wildcard expansion (globbing), quote handling, and variable interpolation.
3. Practice redirection of standard streams, chaining operators, and job control in the background.
4. Solve analytical questions using single-line pipelines (`grep`, `cut`, `sort`, `uniq`, `wc`, `tail`) on an artificial web access log (`access.log`).
5. Implement a robust Bash script that accepts a log file as an argument, validates its existence, and generates statistical failure summaries for individual users.

1.3 Theoretical Notes

A shell acts as a powerful command interpreter linking the user to the kernel. A key concept explored in this work is the **pipeline** (represented by `|`), which routes the standard output of one utility directly into the standard input of another without requiring intermediate temporary files.

The principal text-processing utilities utilized comprise:

- `grep` – searches text files for matching regular expression patterns;
- `cut` – extracts designated fields or columns from text lines;
- `sort` – sorts lines of text alphabetically or numerically;
- `uniq` – filters or counts adjacent duplicate lines;
- `wc` – counts lines, words, and characters in data streams;
- `tail` – outputs the final lines of a target file.

2 IMPLEMENTATION AND PRACTICAL EXECUTION

2.1 Part 1: Move fast, find help

This part explores command identification and system documentation.

- **Built-ins vs Externals:** `cd` and `echo` are built directly into the shell, whereas `ls` is an external executable file. `cd` must be a built-in because a child process cannot modify the working directory of its parent shell process.
- **PATH and Exit Codes:** The system scans directories specified in the `$PATH` variable. An invalid command returns the standard error code **127**.

Figure 1: Figure 1.2 – Output of type and which

2.2 Part 2: What the shell does to your line

Before executing commands, the shell interprets special characters, performs wildcard expansion (globbing), and strips quotes.

Command	**My Prediction**	**Real Output**	**Explanation (if different)**
`echo *.c`	a.c b.c	a.c b.c	Shell expands `*.c` into matching filenames.
`echo file?.txt`	file1.txt file2.txt	file1.txt file2.txt	'?' matches any single character.
`ls alfa beta.txt`	Error / Not found	ls: cannot access...	Spaces split arguments into multiple targets.
`ls "alfa beta.txt"`	alfa beta.txt	alfa beta.txt	Quotes preserve spaces inside the filename.

Table 1: Globbing and quoting predictions table

2.3 Part 3: Redirection, chaining and jobs

Stream redirection allows splitting standard output (`>`) from standard error (`2>`).

- The command sequence `> all.txt 2>&1` accurately merges both streams into a single target file.
- Logical operators such as `;` (sequential execution), `&&` (conditional success), and `||` (conditional failure) enable concise control flow directly inside the shell.

Figure 2: Figure 3.1 – stdout and stderr in separate files

2.4 Part 4: Answer questions with one-liners

Using the artificial log file `access.log`, data extraction was achieved through concise pipelines:

2.4.1 Question 1: Number of Failed Requests

```
grep -c FAIL access.log
```

2.4.2 Question 2: Different Requested Pages

```
cut -d' ' -f5 access.log | sort -u
```

2.4.3 Question 3: Number of Requests for Each Page

```
cut -d' ' -f5 access.log | sort | uniq -c
```

2.4.4 Question 4: Users with the Most Failed Requests

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
```

2.4.5 Question 5: Requests Made by user3

```
grep 'user3' access.log | wc -l
grep 'user3' access.log | grep -c FAIL
```

2.4.6 Question 6: Last Three Failed Requests

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
```

2.4.7 Question 7: Login Shells from /etc/passwd

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

2.4.8 Question 8: Comparing Line Counts

```
ls /etc | wc -l
ls -l /etc | wc -l
```

Explanation: The outputs differ by exactly one because `ls -l` includes a summary header line ('total ...') at the top of the listing.

Figure 3: Figure 4.9 – Full run of oneliners.sh

2.5 Part 5: First Automated Bash Script

A script named `report.sh` was implemented to automate log file verification, argument handling, error emission to standard error (`>&2`), and iterative per-user failure computation.

```
#!/bin/bash

# Usage: ./report.sh access.log
log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")

echo "$total requests,$fails failed"

for user in user0 user1 user2 user3 user4 user5 user6
do
    count=$(grep "$user" "$log" | grep -c FAIL)
    echo "$user:$count failures"
done
```

Figure 4: Figure 5.2 – Running the script on access.log

3 CONCLUSIONS

This laboratory work provided profound insight into the core mechanics of the Linux command-line interface. By investigating pre-execution shell expansions, globbing patterns, and quoting rules, I enhanced my ability to write predictable shell commands.

Furthermore, mastering stream redirection and command pipelines demonstrated that complex data aggregation tasks can be accomplished cleanly without developing heavyweight standalone software. The utilization of standard filters such as `grep`, `cut`, `sort`, and `uniq` proved essential for efficient log analysis. Finally, encapsulating these logical pipelines inside a structured Bash script with explicit parameter validation and proper error handling reinforced robust programming principles for system administration and task automation.